

SnapSense Edge

A Local Multimodal AI System Designed to Be Measured and Optimized for Snapdragon-Powered PCs

Snapdragon® AI Lab Build & Present Challenge — Project Package

1. Executive Project Summary

SnapSense Edge is a local-first multimodal perception system built for Snapdragon-powered Windows PCs, targeting the Snapdragon X Elite / X Plus reference architectures. The system combines three input streams — camera, microphone, and active application context — into a single perception pipeline, and routes the fused output into mode-specific application logic (Study, Interview, Coding).

The problem it addresses: cloud-dependent multimodal AI sends raw sensor data (camera/microphone) across a network boundary before returning a response. This introduces network transmission and round-trip delay and places sensitive media outside the local application boundary. SnapSense Edge is designed around local sensor ingestion, local model execution, and structured local context instead.

How the architecture works: independent vision and speech workers process camera and microphone input in parallel. Their outputs are combined by a deterministic, rule-based context-fusion layer into a structured, memory-resident state. That state — not raw media — is what the local decision logic and UI consume.

Why Snapdragon is relevant: the project's core engineering question is not only "does this pipeline work locally," but "how efficiently does it map onto Snapdragon hardware." The deployment path (Qualcomm AI Hub / Qualcomm AI Runtime / ONNX Runtime

with DirectML) is intended to move candidate models from a CPU baseline toward a profiled, target-compiled deployment on Snapdragon silicon.

What is being measured: the project separates two levels of evaluation — model-level (inference latency, peak memory, compute-unit mapping, output correctness) and application-level (end-to-end response time, preprocessing overhead, frame throughput). As of this package, core perception and mode routing are implemented; Snapdragon hardware profiling is a pending measurement stage, not a completed one.

2. Problem Statement

The project is framed as a systems question, not a performance claim: **what if multimodal AI did not need to leave your PC?**

Four constraints define the engineering problem:

01 — Privacy. Raw sensor data (camera and microphone) can be sensitive. The project treats local processing as an architectural boundary intended to reduce transmission of raw media — not as an absolute privacy guarantee.

02 — Latency. Network round trips can affect live, interactive use. Model and application latency need to be measured rather than assumed.

03 — Compute. Vision and speech workloads can compete for general-purpose compute. The target is to evaluate how the combined workload maps onto dedicated Snapdragon acceleration (NPU/CPU split), rather than to assume that mapping in advance.

04 — Proof. Running an application on an Arm-based Snapdragon PC is not, by itself, evidence of hardware optimization. Compute mapping and performance claims must be backed by profiling

evidence — compilation, profiling, and validation as distinct, separately-recorded steps.

Central engineering question:

"Can we build a useful multimodal AI pipeline locally and measure how efficiently the target hardware runs it?"

3. System Architecture

3.1 Data flow

Camera (OpenCV, 640×640 / 320×320 target input)

→ Vision Worker (RTMDet / YOLOv4-Tiny)

→ Visual Context

Microphone (PortAudio, 16 kHz mono Float32 PCM stream)

→ Speech Worker (Whisper-Tiny)

→ Speech Context

Active Application (current mode / state)

↘

Visual Context + Speech Context + Active App State

→ Deterministic Context Fusion

→ Local Decision

→ UI / Action

Vision and speech are handled by **independent workers** — they run without waiting on each other — and their outputs are combined only at the context-fusion stage. This is described in the specification as decoupled, asynchronous inference feeding a centralized deterministic context layer.

3.2 Why deterministic context fusion, not another model

The specification's design choice is to use rule-based, deterministic fusion logic instead of adding an intermediate neural fusion model. Two stated reasons:

- **Transparent state inspection.** Because fusion is rule-based, the resulting structured state can be inspected directly rather than interpreted through another model's internal representation.
- **A clear boundary for performance profiling.** Deterministic fusion keeps the model-level workload (vision, speech) cleanly separated from the application logic that consumes it, which supports the project's model-level vs. application-level measurement split (Section 8).

3.3 Example structured context

The specification defines the context state using fields such as:

Field	Example from project specification
<code>active_mode</code>	<code>coding</code>
<code>speech_context</code>	transcript + confidence + processing time
<code>visual_context</code>	detected entities + bounding-box count + processing time
<code>runtime_telemetry</code>	target device + compute layer

This is a memory-resident structured state, not raw sensor data retained as the application's permanent state.

4. Application Modes

SnapSense Edge uses **one shared perception pipeline** across three application modes. The mode changes which fields of the fused context matter and how the local decision logic interprets them — it does not change the underlying vision/speech pipeline.

Study Mode. Input: speech query + notebook capture. Local action: concept hints and active-recall style prompts.

Interview Mode. Input: spoken answer + visual/pose context. Local action: pacing and delivery-related context for practice.

Coding Mode. Input: spoken query + active application context. Local action: local error/context extraction for targeted coding hints.

Defined application principle (from the specification): one perception pipeline, structured context, mode-specific logic. This is explicitly framed as avoiding the need to build three unrelated applications — the modes are different consumers of the same fused context, not three separate systems.

5. Model Selection

The specification treats model selection as an empirical evaluation parameter, to be resolved through the deployment/measurement workflow (Section 6) rather than decided in advance.

Model	Role	Approx. Parameters	Why Considered	Deployment / Measurement Focus
Whisper-Tiny	Automatic speech recognition (speech perception candidate / baseline)	Not specified in the project specification	Initial candidate for the speech worker	Latency, memory, and correctness once compiled to the Snapdragon target
RTMDet	Real-time object detection (primary vision candidate)	Not specified in the project specification	Initial candidate for the vision worker	Latency, memory, and compute mapping (NPU + CPU per Qualcomm reference profile) once compiled and profiled
YOLOv4-Tiny	Compact detection (vision comparison candidate)	Not specified in the project specification	Comparison candidate against RTMDet for the vision worker	To be evaluated against RTMDet on the same latency/memory/correctness criteria

Selection criteria defined in the specification:

- **Task fit** — does the model provide the information required by the active application mode?
- **Deployment** — can the model be prepared for the intended Snapdragon deployment path?
- **Latency + memory** — what happens to inference time and memory after deployment?

- **Correctness** — does the target (compiled) artifact preserve the expected model output relative to baseline?

The specification also lists model size, catalog availability, and precision/deployment characteristics as additional selection considerations. **No final model choice is declared** — status for all three candidates is "Initial candidate," and any final selection should remain tied to the measured deployment stage (Section 6-7).

6. Snapdragon Optimization Workflow

The specification defines a six-stage workflow from model/task decision to integrated target deployment. The stated distinction driving this workflow: **running a model on a Snapdragon PC is not, by itself, proof of Snapdragon optimization** — so compilation, profiling, and verification are kept as separate, individually-recorded steps.

Stage	Description	What is measured
01 — SELECT	Choose model + task constraints.	Candidate model and the task/mode it serves.
02 — BASELINE	Measure unoptimized execution on the defined baseline runtime.	Unoptimized latency/memory as a reference point before any target-specific compilation.
03 — COMPILE	Prepare a target graph through Qualcomm AI Hub for the intended Snapdragon target.	Whether the model compiles successfully to the target graph.
04 — PROFILE	Run target profiling and record available execution/memory information.	On-target latency, memory, and available compute-mapping information.
05 — VALIDATE	Check output parity between baseline and target artifacts.	Numerical correctness of the compiled target artifact vs. baseline.
06 — INTEGRATE	Bind validated artifacts into the SnapSense application loop.	Whether the validated, target-compiled model functions correctly inside the running application.

Per the current implementation status (Section 11), Stage 03 (AI Hub compilation) is in progress; Stages 04–06 are not yet measured or completed.

7. Qualcomm AI Hub Reference Evidence

⚠️ QUALCOMM REFERENCE PROFILES — NOT SNAPSENSE RESULTS

The specification records the following **published Qualcomm AI Hub reference figures**, measured on Snapdragon X Elite reference hardware (CRD):

Model	Target	Inference time	Peak memory	Compute
RTMDet	Snapdragon X Elite CRD	~15.3 ms	~27 MB	NPU + CPU
FFNet-40S	Snapdragon X Elite CRD	~10.6 ms	~8 MB	NPU
RTMPose-Body2d	Snapdragon X Elite CRD	~1.84 ms	~20 MB	NPU

These are external reference profiles published by Qualcomm on Snapdragon X Elite reference hardware. They are not measurements of the SnapSense Edge application, and none of the models in this table other than RTMDet are part of SnapSense's model candidates (FFNet-40S and RTMPose-Body2d are cited only as additional context on what the Qualcomm AI Hub profiling environment exposes).

The specification uses these figures to establish that lightweight models *can* have low-millisecond inference profiles on the target reference platform — it does not use them to predict or imply SnapSense's own numbers. The specification also explicitly distinguishes **model inference latency** from **full application**

response time; the reference table above describes only the former, for models run in Qualcomm's own environment.

8. Evaluation Methodology

SnapSense separates evaluation into two levels so that model-level behavior is never confused with full application behavior.

A. Model-level evaluation

Isolates model/runtime behavior:

- Steady-state inference latency
- Peak working-set memory
- Model size
- Compute-unit mapping (CPU / NPU, once verified through profiling)
- Output correctness (numerical parity between baseline and target artifacts)

B. Application-level evaluation

Measures sensor-to-UI behavior:

- Preprocessing overhead
- Model inference (as consumed inside the application loop)
- Context fusion time
- Decision logic time
- UI / action response
- Total end-to-end response time
- Frame throughput

Current status of each metric

Metric	Baseline	Snapdragon target	Status
Vision latency	To be measured	To be measured	Pending profiling
Speech latency	To be measured	To be measured	Pending profiling
Peak memory	To be measured	To be measured	Pending profiling
Compute mapping	CPU baseline	Target compute mapping to be verified	Pending trace
Output correctness	Baseline output	Numerical parity check	Validation stage
End-to-end latency	To be measured	To be measured	Application stage

Fair-comparison protocol (as proposed in the specification)

- Identical recorded inputs used for both baseline and target comparisons
- First five inference cycles discarded as warm-up
- Target SoC, OS build, and runtime driver version documented alongside results

This protocol is described in the specification as **proposed** — it has not yet been executed, and results should remain labelled as a proposed methodology until it is.

9. Privacy Architecture

The specification treats privacy as a data-boundary decision, not a blanket guarantee.

Local sensor ingestion. Camera and microphone processing stay inside the local runtime; the specification describes transient, in-memory handling as the intended architecture. No disk-caching behavior for raw sensor data is stated in the specification beyond this in-memory design.

Bounded state. The system passes compact structured context (Section 3.3) to the decision layer rather than treating raw media as the application's permanent state.

Network boundary. The specification describes network transmission as disabled by default for the local perception path.

Threat-model limit. Local execution reduces exposure in transit, but the application still depends on host OS permissions and endpoint integrity. The specification does not claim complete or absolute privacy — local processing reduces the transmission exposure of raw media; it does not guarantee security against host-level compromise.

10. User Journey

Scenario: Interview Mode, coding-topic question

Step	Description
01 — Sensor ingestion	The user is in Interview Mode and speaks; relevant visual/pose context is observed by the camera simultaneously.
02 — Parallel inference	Whisper-Tiny processes the audio buffer while the vision pipeline extracts visual context — these run independently.
03 — Context fusion	The two independent outputs are combined into a structured in-memory state, e.g. <code>active_mode: interview</code> , plus the transcript/confidence from speech and detected entities from vision.
04 — Local decision	The mode and extracted context are evaluated by the application's local decision logic (in Interview Mode, this targets pacing and delivery-related context).
05 — UI feedback	The desktop interface presents the resulting contextual hint and available telemetry (e.g., processing time per stream).

This walkthrough follows the general user-journey structure defined in the specification (Section 09 of the source document), applied to a concrete interview-mode interaction.

11. Implementation Status

Milestone	Work described in project specification	Status
M1 — Core perception	OpenCV camera pipeline; PortAudio speech ingestion; in-memory buffers.	IMPLEMENTED
M2 — Context + modes	JSON context contracts; Study/Interview/Coding routing; developer UI.	IMPLEMENTED
M3 — AI Hub compilation	Candidate model export; AI Hub connection; target compilation jobs.	IN PROGRESS
M4 — Profiling	Physical Snapdragon profiling; layer breakdown; memory; parity checks.	TO BE MEASURED
M5 — Final integration	Optimized artifacts integrated; end-to-end comparison; final documentation.	PLANNED

This table is intentionally load-bearing for the presentation: the project does not present future Snapdragon measurements as if they already exist. Core perception and mode routing are built and working locally; the Snapdragon-specific optimization work (compilation → profiling → validation → integration) is the part still in progress or pending.

12. Known vs. Unknown

Known / Confirmed	Still To Test / Measure
Modular vision/speech architecture with independent workers	Actual Whisper-Tiny latency on physical Snapdragon hardware
Deterministic context fusion design and structured state schema	Concurrent memory behavior when vision + speech run together
Mode routing across Study / Interview / Coding	Operator fallback distribution during NPU/CPU compilation
Qualcomm AI Hub deployment workflow (6 stages) defined	End-to-end application latency (sensor to UI) on target hardware
Evaluation framework (model-level vs. application-level) defined	Whether RTMDet or YOLOv4-Tiny is the better vision candidate once profiled
Qualcomm's own published reference profiles for RTMDet, FFNet-40S, RTMPose-Body2d	Output-parity (baseline vs. target) results for SnapSense's own compiled models

13. Technical Contribution

The specification does not claim a new foundational machine-learning architecture. The contribution is positioned as practical edge-AI **systems engineering**, concentrated in:

- **Co-design of application and hardware target.** Separate sensory models (vision, speech) run independently, and their outputs are merged through deterministic context handling designed with the target deployment in mind.

- **Snapdragon alignment.** A defined transition path from a general local baseline toward a Qualcomm-targeted, profiled deployment.
- **Measurement discipline.** Model-level behavior (inference latency, memory, compute mapping) is kept separate from end-to-end application behavior, reducing the risk of conflating the two.
- **Transparent scope.** Reference benchmarks (Qualcomm's), baseline measurements (SnapSense's own, pending), and future target-device evaluation (pending) are kept in clearly separate evidence categories.

No claim of scientific/ML novelty is made beyond this — the specification frames SnapSense as applied systems work, not new model research.

14. Limitations

- **Snapdragon measurements are pending.** Vision latency, speech latency, peak memory, compute mapping, and end-to-end latency are all currently "to be measured" (Section 8) — no final performance numbers exist yet.
- **Model selection may change after profiling.** RTMDet and YOLOv4-Tiny are both still "initial candidate" status; the eventual choice depends on measured latency/memory/correctness, not a decision made now.
- **Qualcomm's published numbers are not SnapSense's results.** The RTMDet/FFNet-40S/RTMPose-Body2d reference table (Section 7) describes Qualcomm's own profiling environment, not this application.
- **Local processing does not equal absolute privacy.** It reduces transmission exposure of raw sensor data but does not protect against host OS compromise or endpoint-level threats.

- **Not a replacement for cloud-scale reasoning.** SnapSense is scoped as a multimodal perception/state pipeline, not a general-purpose large language model.
 - **NPU execution is not yet claimed.** Compute-unit mapping (CPU vs. NPU) for SnapSense's own models is a profiling result to be obtained, not an assumption made in advance.
-

15. Competition Pitch — Slide Structure (12 slides)

Slide 1 — Title

Main message: Introduce the project and its framing question. **Key points:**

- SnapSense Edge — local multimodal AI for Snapdragon-powered PCs
- Snapdragon® AI Lab Build & Present Challenge
- Team / student name(s), date
- Framing question up front

Suggested visual: Simple pipeline icon (camera + mic → gear → screen), no illustration of hardware not yet tested.

Slide text:

SnapSense Edge A Local Multimodal AI System Designed to Be Measured and Optimized for Snapdragon-Powered PCs "If AI is running locally, can we actually measure how well it uses the hardware?"

Slide 2 — Problem

Main message: Cloud-dependent multimodal AI has structural costs; the project starts from a systems question. **Key points:**

- Network transmission + round-trip delay in cloud-dependent paths
- Sensitive media crosses the local application boundary
- Four constraints: Privacy, Latency, Compute, Proof
- Central question stated explicitly

Suggested visual: Two-path diagram: Cloud-Dependent Path vs. Local-First Path.

Slide text:

Can we build a useful multimodal AI pipeline locally — and measure how efficiently the target hardware runs it? Constraints: Privacy · Latency · Compute · Proof

Slide 3 — Application

Main message: One perception pipeline, three targeted workflows.

Key points:

- Study Mode — speech + notebook capture
- Interview Mode — spoken answer + visual/pose context
- Coding Mode — spoken query + active app context
- Shared pipeline, mode-specific logic

Suggested visual: Three mode icons branching from one shared pipeline box.

Slide text:

One perception pipeline. Three modes: Study · Interview · Coding. Same sensors, same fusion logic — different application context.

Slide 4 — System Architecture

Main message: How raw sensor signals become deterministic local state. **Key points:**

- Camera → Vision worker (RTMDet / YOLOv4-Tiny) → Visual Context
- Microphone → Speech worker (Whisper-Tiny) → Speech Context
- Deterministic Context Fusion → Local Decision → UI/Action
- Independent, asynchronous workers

Suggested visual: The architecture block diagram from the specification (Section 03).

Slide text:

Vision and speech run independently. Context fusion is deterministic, not another model.

Slide 5 — Why Deterministic Fusion

Main message: A design choice, not a limitation. **Key points:**

- Rule-based fusion instead of a second neural model
- Transparent, inspectable state
- Clear boundary for profiling model-level vs. application-level performance
- Structured fields: `active_mode`, `speech_context`, `visual_context`, `runtime_telemetry`

Suggested visual: Example JSON-style context snippet.

Slide text:

Deterministic context fusion: a transparent, inspectable boundary — built for measurement, not just function.

Slide 6 — Model Candidates

Main message: Model selection is an open, empirical question. **Key points:**

- Whisper-Tiny — speech (initial candidate)
- RTMDet — vision (initial candidate)
- YOLOv4-Tiny — vision comparison (initial candidate)
- Selection criteria: task fit, deployment, latency + memory, correctness

Suggested visual: Simple candidate table.

Slide text:

No model is declared "final" yet. Selection is tied to measured deployment results.

Slide 7 — Snapdragon Deployment Workflow

Main message: Six explicit stages from model choice to integration.

Key points:

- SELECT → BASELINE → COMPILE → PROFILE → VALIDATE → INTEGRATE
- Qualcomm AI Hub used for target compilation
- Parity check between baseline and target artifacts
- "Running on Arm is not proof of optimization"

Suggested visual: Six-stage horizontal workflow diagram.

Slide text:

Compilation, profiling, and validation are kept as separate, evidenced steps — not assumed.

Slide 8 — Qualcomm Reference Evidence

Main message: What Qualcomm's own profiling shows — clearly separated from this project's results. **Key points:**

- RTMDet: ~15.3 ms / ~27 MB / NPU + CPU
- FFNet-40S: ~10.6 ms / ~8 MB / NPU
- RTMPose-Body2d: ~1.84 ms / ~20 MB / NPU
- Snapdragon X Elite CRD reference hardware
- Explicitly labelled: not SnapSense results

Suggested visual: Reference table with a visible "REFERENCE ONLY" banner.

Slide text:

QUALCOMM REFERENCE PROFILES — NOT SNAPSENSE RESULTS. Shown to illustrate what the profiling environment can expose.

Slide 9 — Evaluation Methodology

Main message: Two levels of measurement, kept separate. **Key points:**

- Model-level: latency, peak memory, compute mapping, correctness
- Application-level: preprocessing, fusion, decision, UI, end-to-end latency
- Same inputs, same preprocessing, baseline vs. target
- First 5 cycles discarded as warm-up (proposed protocol)

Suggested visual: Two-column table (Model-level vs. Application-level).

Slide text:

Model behavior is never confused with full application behavior.

Slide 10 — Implementation Status

Main message: Honest state of the build. **Key points:**

- M1 Core perception — IMPLEMENTED
- M2 Context + modes — IMPLEMENTED
- M3 AI Hub compilation — IN PROGRESS
- M4 Profiling — TO BE MEASURED
- M5 Final integration — PLANNED

Suggested visual: Status table with color-coded (not exaggerated) status labels.

Slide text:

We do not present future Snapdragon measurements as if they already exist.

Slide 11 — Engineering Boundaries

Main message: What this project deliberately does not claim. **Key points:**

- No invented benchmarks or synthetic frame rates
- No unverified NPU utilization claims
- No borrowed success (Qualcomm numbers stay separate)
- No absolute privacy guarantee
- No general-LLM / cloud-reasoning claim

Suggested visual: Simple "does / does not claim" two-column list.

Slide text:

Hardware claims are backed by profiling. External reference data stays separate.

Slide 12 — Closing

Main message: Restate the project's core discipline and next step.

Key points:

- Local-first multimodal perception, Snapdragon-targeted
- Core pipeline built and working; profiling stage next
- Evidence-driven, not assumption-driven
- Call back to the framing question

Suggested visual: Same pipeline diagram as Slide 1, now with "measurement in progress" annotation.

Slide text:

Build locally. Measure honestly. Optimize for the edge.

16. Final Closing

SnapSense Edge is a local-first multimodal perception system targeted for Snapdragon-powered Windows PCs. Its core pipeline combines video, audio, and active application context, runs independent perception stages, fuses their outputs into structured state, and routes that state into local application logic across three modes.

Dimension	Project specification
System objective	Local-first multimodal perception system targeted for Snapdragon PCs
Inputs	Web camera + microphone + active application context
Core mechanism	Decoupled asynchronous inference → deterministic context fusion → local decision/UI
Initial models	Whisper-Tiny + RTMDet / YOLOv4-Tiny
Deployment path	Qualcomm AI Hub / Qualcomm AI Runtime / ONNX Runtime with DirectML
Primary evaluation	Baseline behavior vs. target Snapdragon deployment — latency, memory, compute mapping
Evidence rule	Hardware claims backed by profiling; external reference data kept separate

The intended final result is not a performance number chosen in advance. It is a documented experiment showing what changes when the same AI task moves from a defined baseline toward a measured Snapdragon deployment.

Build locally. Measure honestly. Optimize for the edge.